

تطبيق الويست السودانية

PRD + TAD — وثيقة المتطلبات والتصميم الفني

إصدار 1.0 | AE / Khatwa Tech | Flutter

1. نظرة عامة على المنتج

تطبيق جوال (Flutter) للعبة الويست السودانية التقليدية، يدعم وضعين أساسيين منذ الإصدار الأول: اللعب الفردي ضد بوتات بالذكاء الاصطناعي، واللعب الجماعي عبر الشبكة مع 4 لاعبين حقيقيين (فريقين من اثنين). الهدف هو نقل تجربة اللعب الاجتماعية السودانية التقليدية إلى تجربة رقمية أصيلة تحافظ على قواعد اللعبة الدقيقة، خصوصاً نظام المزايدة (التسمية) والباص.

1.1 الجمهور المستهدف

- اللاعبون السودانيون والمهتمون بألعاب الورق الشعبية في الخليج والشرق الأوسط
- مستخدمو الجيل الأول من تطبيقات الورق الذين يريدون بديلاً يحترم القواعد المحلية الدقيقة (بخلاف تطبيقات الويست العالمية)
- عائلات وجماعات أصدقاء يريدون اللعب عن بُعد بنفس قواعد اللعب على الطاولة

1.2 المبادئ التوجيهية

- القواعد أولاً: المحرك يجب أن يطابق قواعد الويست السودانية بدقة (التسمية تبدأ من مراجعة الأوراق، الباص، من يسمي يبدأ اللعب، نظام النقاط الخاص)
- لا انتظار غير ضروري: تسلسل اللعب يجب أن يتقدم تلقائياً دون توقف أو تعليق
- دعم اللغة العربية RTL بالكامل في الواجهة
- بنية قابلة للتوسع لإضافة أوضاع لعب مستقبلية (دبل/ريدبل، بطولات، تصنيف لاعبين)

2. نطاق الإصدار الأول (MVP)

2.1 داخل النطاق

- وضع اللعب الفردي: لاعب واحد + 3 بوتات (شريك وخصمان)
- وضع متعدد اللاعبين: 4 لاعبين حقيقيين عبر الشبكة، فريقين من اثنين متقابلين
- نظام مزايدة كامل (7-13) + خيار الباص + تمرير التسمية للشريك أو الخصم
- اختيار الكوز (4 بذرات + بلا كوز)
- محرك لعب الأوراق الكامل (13 حول، اتباع البذرة، القمع بالكوز)
- نظام نقاط دقيق: نجاح = + عدد الإيادي، فشل = - التسمية، الطرف الآخر = 0 أو + إياديه عند فشل الخصم
- نهاية المباراة عند الوصول لنقاط الفوز المتفق عليها (افتراضي: 25 أو -25)
- واجهة عربية RTL بالكامل

2.2 خارج النطاق (إصدارات لاحقة)

- نظام الدبل / الريدبل (مضاعفة المزايدة)
- غرف لعب خاصة بكود دعوة + دردشة صوتية/نصية بين اللاعبين
- نظام تصنيف (ranking) وبطولات
- اللعب دون اتصال بالإنترنت (offline) ضد بوتات فقط — قد يدخل النطاق إذا اعتُبر منخفض التكلفة
- الاشتراكات أو الشراء داخل التطبيق

3. مواصفات قواعد اللعبة (مرجع المحرك)

3.1 الإعداد

1. 4 لاعبين في فريقين متقابلين (شمال-جنوب ضد شرق-غرب)
2. توزيع 52 ورقة بالكامل، 13 ورقة لكل لاعب
3. كل لاعب يرى أوراقه فقط قبل أي مزايده

3.2 مرحلة المزايده (التسمية)

4. يبدأ لاعب محدد (دوار أو يحدده النظام) بمراجعة أوراقه واختيار: تسمية عدد بين 7 و 13 + اختيار الكوز (أو بلا كوز)، أو الباص
5. إذا سمى اللاعب: فريقه يصبح صاحب التسمية (bidder)، وهو أول من يلعب ورقة في الحول الأول
6. إذا اختار الباص: يُعرض الفرصة على شريكه؛ إذا سمى الشريك، فريقهما يصبح صاحب التسمية وشريكه يبدأ اللعب
7. إذا باص اللاعب وشريكه أيضاً: تنتقل الفرصة للخصم (المحرك الحالي للنموذج يبسطها للذكاء الاصطناعي)؛ من يسمي من الخصم يبدأ اللعب
8. إذا باص الجميع: تُعاد عملية التوزيع للجولة

3.3 مرحلة اللعب

9. من سمى (أو شريكه) يبدأ بلعب أي ورقة من يده من بذرة الكوز (الزامي للورقة الأولى من الحول الأول)
10. بقية اللاعبين يتبعون بنفس البذرة المطلوبة إن توفرت في يدهم؛ إن لم تتوفر، يجوز القطع بالكوز أو رمي أي ورقة أخرى
11. صاحب أعلى ورقة من البذرة المطلوبة يفوز بالحول، إلا إذا وُجد كوز فيكون الكوز الأعلى هو الفائز
12. الفائز بالحول يبدأ الحول التالي
13. تتكرر العملية 13 مرة حتى انتهاء جميع الأوراق

3.4 حساب النقاط (دقيق - تم تأكيده مع المستخدم)

الحالة	النتيجة
الفريق المسمّى نجح (الإيادي ≤ التسمية)	+ عدد الإيادي المُحصّلة
الفريق المسمّى فشل (الإيادي > التسمية)	- قيمة التسمية كاملة
الفريق الآخر (لم يسم) — والمسمّى نجح	0 (بدون نقاط)
الفريق الآخر (لم يسم) — والمسمّى فشل	+ عدد الإيادي التي حصلها

أمثلة محسوبة:

- سمى الفريق 7 وجاب 7 إيادي → 7+ / الخصم لم يسم → 0
 - سمى الفريق 8 وجاب 10 إيادي → 10+ / الخصم → 0
 - سمى الفريق 9 وجاب 8 إيادي (فشل) → 9- / الخصم جاب 5 → 5+
 - سمى الفريق 11 وجاب 10 إيادي (فشل) → 11- / الخصم جاب 3 → 3+
- الفريق الفائز بالمباراة هو من يصل أولاً إلى عتبة النقاط المتفقة (افتراضي: $25 \pm$)، أو من يجمع كل الـ 13 حول في جولة واحدة (كابوت) حسب القرار المعماري لاحقاً.

4. التصميم المعماري (Technical Architecture)

4.1 المكس التقني المقترح

الطبقة	التقنية المقترحة
Client Framework	Flutter (Dart) — يدعم iOS/Android من قاعدة كود واحدة
إدارة الحالة (State)	Riverpod (متوافق مع خبرتك السابقة في Food Truck Driver)
منطق اللعبة (Game Engine)	حزمة Dart منفصلة (pure logic, لا تعتمد على Flutter widgets) — قابلة لإعادة الاستخدام في الخادم لاحقاً
الاتصال الفوري (Multiplayer)	WebSocket (خادم Node.js/Express + ws، أو Firebase Realtime Database / Firestore لتسريع MVP)
المصادقة (Auth)	Firebase Auth أو نظام JWT بسيط (تسجيل دخول بالهاتف/Google)
التخزين السحابي	Firestore (نتائج، إحصائيات لاعبين) — يتماشى مع AWS/OCI خبرتك إذا فُضِّلَ EC2 + RDS
الإشعارات	Firebase Cloud Messaging (دعوة لاعب لجولة، دورك في اللعب)

4.2 البنية المعمارية للعميل (Client Architecture)

يُفترض استخدام Clean Architecture مبسطة مع 3 طبقات:

- 14. Presentation Layer: شاشات (Flutter + Widgets (BiddingScreen, GameTableScreen, ScoreboardScreen)
- 15. Domain Layer: حزمة west_engine المستقلة — تحتوي قواعد التوزيع، التسمية، اللعب، حساب النقاط (Pure Dart، قابلة للاختبار بدون واجهة)
- 16. Data Layer: GameRepository يوفر تجريد بين المنطق ومصدر البيانات (remote أو local bot engine) (multiplayer socket)

4.3 تمثيل حالة اللعبة (Core Models)

- { Card { rank, suit
- { <PlayerHand { cards: List<Card
- { BidState { biddingTeam, bidValue, trumpSuit, isPassed
- { TrickState { playedCards: List<PlayedCard>, leadSuit, winnerIndex
- { RoundState { tricksWon: Map<Team,int>, currentTrickNumber, firstPlayerIndex
- { MatchState { usScore, themScore, roundHistory, gameMode
- { GameMode enum { singlePlayerVsBots, onlineMultiplayer

4.4 منطق تسلسل اللعب (Turn Sequencing)

نقطة الفشل الأساسية في النموذج التجريبي (الويب) كانت توقف التسلسل عندما يكون اللاعب التالي بوتاً وسط سلسلة من البوتات. التصميم الموصى به للتطبيق:

- استخدام دالة واحدة advanceTurn() تُستدعى بعد كل ورقة تُلعب — تتحقق من اللاعب التالي وتُشغل تلقائياً منطق البوت أو تنتظر إدخال المستخدم، بدون تفرّع متعدد للحالات
- في الوضع الجماعي: الخادم هو المصدر الوحيد للحقيقة (source of truth) لتسلسل الأدوار، والعميل يستقبل turnState عبر WebSocket ويُحدّث الواجهة تفاعلياً

- استخدام Future/async بدلاً من setTimeout المتداخل (الذي تسبب بالتعقيد في إصدار الويب)

4.5 خادم اللعب الجماعي (Multiplayer Server) — مخطط مستقبلي

17. Room Manager: ينشئ غرفة 4 مقاعد، يدير حالة الانضمام/الانتظار
18. Game Session: نسخة من west_engine تعمل على الخادم لكل غرفة نشطة، تضمن عدم الغش (لا أحد يرى أوراق الآخر)
19. Sync Protocol: رسائل WebSocket مبسطة: bid, pass, playCard, gameStateUpdate
20. Reconnection Handling: حفظ حالة الجولة لمدة محددة (مثلاً 60 ثانية) لإعادة الاتصال عند انقطاع الشبكة

5. مخطط الشاشات (UX Flow)

- Splash / شاشة البداية
- Main Menu: لعب فردي / لعب جماعي / الإحصائيات / الإعدادات
- Mode Select: اختيار وضع اللعب (وحدة بوتات أو إنشاء/الانضمام لغرفة)
- Lobby Screen (للوضع الجماعي): قائمة المقاعد + كود الدعوة + زر بدء اللعب
- Hand Preview Screen: عرض الأوراق 13 قبل المزايدة
- Bidding Screen: أزرار 13-7 + باص + اختيار الكوز
- Game Table Screen: الشاشة الأساسية — يد اللاعب + منطقة اللعب + عداد الحول + نقاط الفريقين
- Round Summary Screen: تفاصيل حساب النقاط بعد كل جولة
- Match End Screen: الفريق الفائز + خيار إعادة اللعب

6. خارطة الطريق المقترحة

المرحلة	المخرجات
Phase 1	بناء حزمة (Pure Dart) (west_engine) + اختبارات وحدة شاملة لكل قاعدة لعب وحساب نقاط
Phase 2	واجهات Flutter لوضع اللعب الفردي (ضد بوتات) بالكامل + دمج المحرك
Phase 3	تصميم بروتوكول الاتصال + خادم WebSocket بسيط (Node.js) لدعم 4 لاعبين حقيقيين
Phase 4	دمج الوضع الجماعي في التطبيق + اختبار الشبكة وإعادة الاتصال
Phase 5	صقل الواجهة (RTL، تأثيرات حركية، صوتيات) + نشر تجريبي (Closed Testing) على (Google Play)

7. الخطوات التالية

بانتظار موافقتك على هذا التصميم، الخطوة التالية المقترحة هي بناء حزمة west_engine بالكامل (Pure Dart) مع اختبارات وحدة (unit tests) تغطي كل سيناريوهات الحساب التي تم تأكيدها أعلاه، ثم البدء بشاشات الوضع الفردي.